

Generate Permutations

Generate Permutations

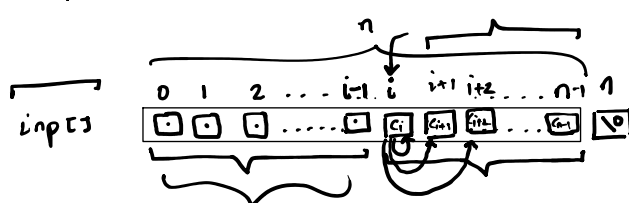
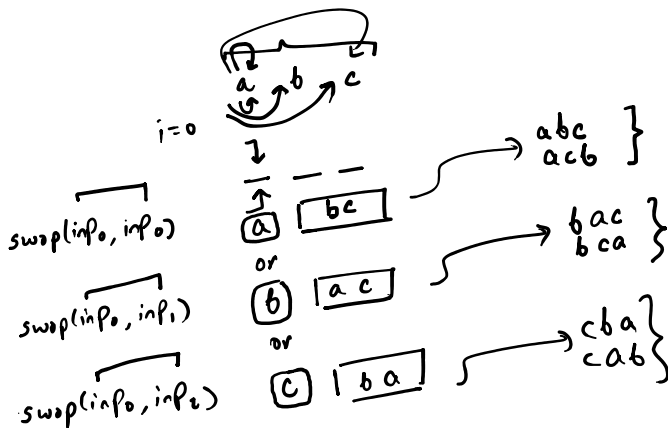
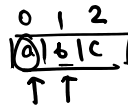
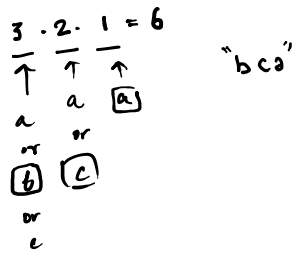
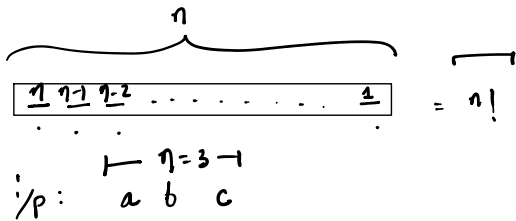
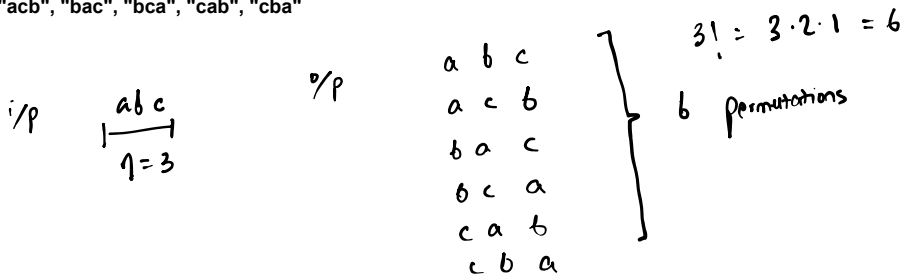
Given a string, design a recursive algorithm to generate all of its **permutations**.

A **permutation** of a given string is a **rearrangement** of its characters.

Example

Input : "abc"

Output : "abc", "acb", "bac", "bca", "cab", "cba"



if (i == n) { inpt[i] == '\0'
 cout << inpt;
 ret;
 }

$\text{swap}(inpt_i, inpt_i) \rightarrow f(i+1)$
 $\text{swap}(inpt_i, inpt_{i+1}) \rightarrow f(i+1)$
 $\text{swap}(inpt_i, inpt_{i+2}) \rightarrow f(i+1)$
 $\vdots$
 $\text{swap}(inpt_i, inpt_{n-1}) \rightarrow f(i+1)$

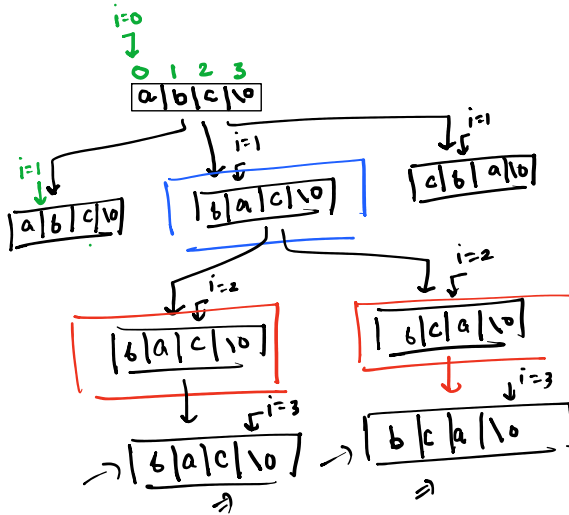
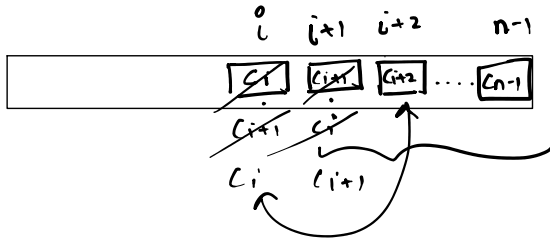
$\text{swap}(inpt_i, inpt_j) \rightarrow f(i+1)$
 $i \leq j \leq n-1$

$f(i)$ = it is a fn that takes decisions for idx i to n-1
 to solve this just decide p. the ith idx

to solve this just decide
for the i th idx

swap (inp_i, inp_{n-1}) $\rightarrow f(i+1)$

init $i=0$



Generate Unique Permutations

Given a string, design a recursive algorithm to **generate** all of its **unique permutations**.

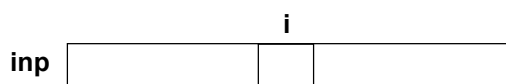
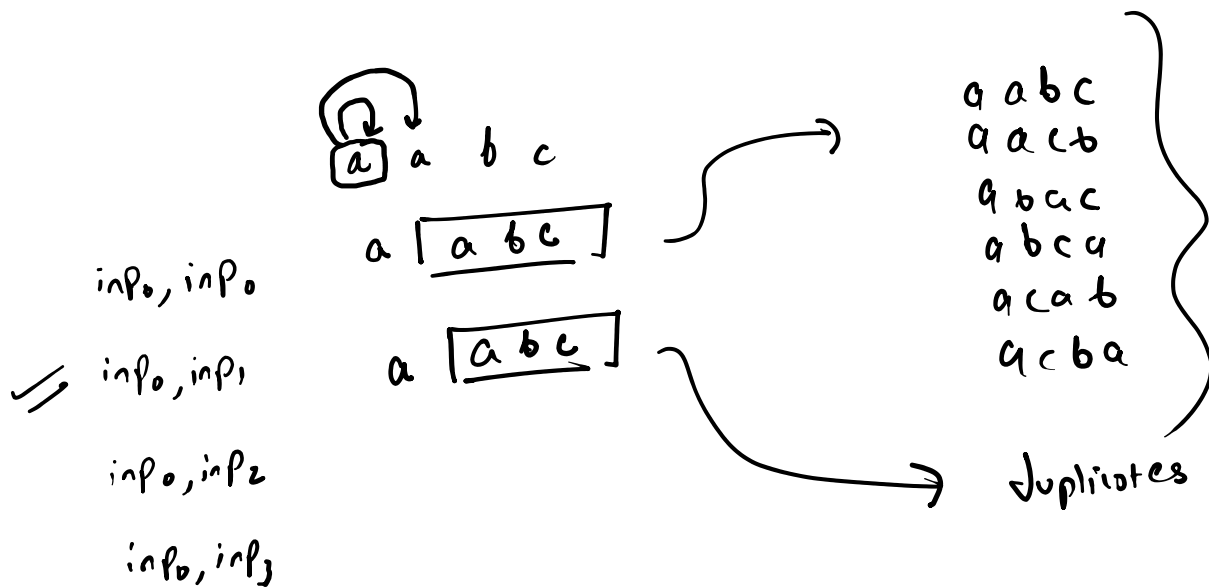
A **permutation** of a given string is a **rearrangement** of its characters.

note : generate the permutations without using any extra space.

Example

Input : "aabc"

Output : "aabc", "aacb", "baac", "baca", "bcaa", "caab", "caba", "cbaa"

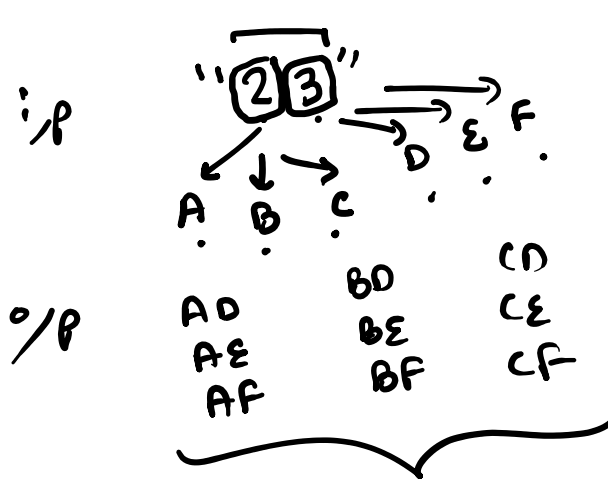


[HW] Letter Combination of a Phone Number

Letter Combination of a Phone Number

Given a **phone keypad**, and a string representing a **n-digit** number write a program to **generate all the words** possible upon **pressing** the n-digits on the phone keypad.

"23"
"ad"
"ae"
"af"
"bd"
"be"
"bf"
"cd"
"ce"
"cf"



[HW] Construct Smallest Number From DI String

Construct Smallest Number From DI String

Given a 0-indexed string **pattern** of length **n** consisting of characters 'I' meaning increasing and 'D' meaning decreasing. A 0-indexed string **num** of length **n+1** is created using the following conditions -

- num** consists of the digits '1' to '9' where each digit is used at most once
- if **pattern** [i] == 'I', then **num** [i] < **num** [i+1]
- if **pattern** [i] == 'D', then **num** [i] > **num** [i+1]

Design an algorithm to return the lexicographically smallest possible string **num**.

Example

	0	1	2	3	4	5	6	7	8
pattern	I	I	I	D	I	D	D	D	
num	1	2	3	5	4	9	8	7	6

	0	1	2	3	4
pattern	D	I	I	D	
num					

Handwritten notes and diagrams illustrating the algorithm:

Diagram showing the pattern and the resulting number:

```

pattern: 0 1 2 3 4 5 6 7 8
          I I I D I D D D
num:      1 2 3 5 4 9 8 7 6
  
```

Diagram showing the pattern and the resulting number for the example:

```

pattern: 0 1 2 3 4
          D I I D
num:      5 4 3 2 1
  
```

Handwritten notes showing the logic for the example:

1 → false true
 2 → false true
 3 → false true
 4 → false true
 5 → false
 6 → false
 7 → false
 8 → false
 9 → false

pattern

--	--	--

num

--	--	--

Permutation Sum

Permutation Sum

Given an array of **distinct** integers **candidates** and a target integer **target**, design a algorithm to find all the **permutations of candidates** that sum up to the target. You may print the permutations in **any order**.

The **same** number may be chosen from candidates an **unlimited number of times**.

Example

Input

target = 7	0	1	2	3
candidates	2	3	5	7

Output

2

2

3

2

5

2

2

5

2

2

5

2

7

~~~~~

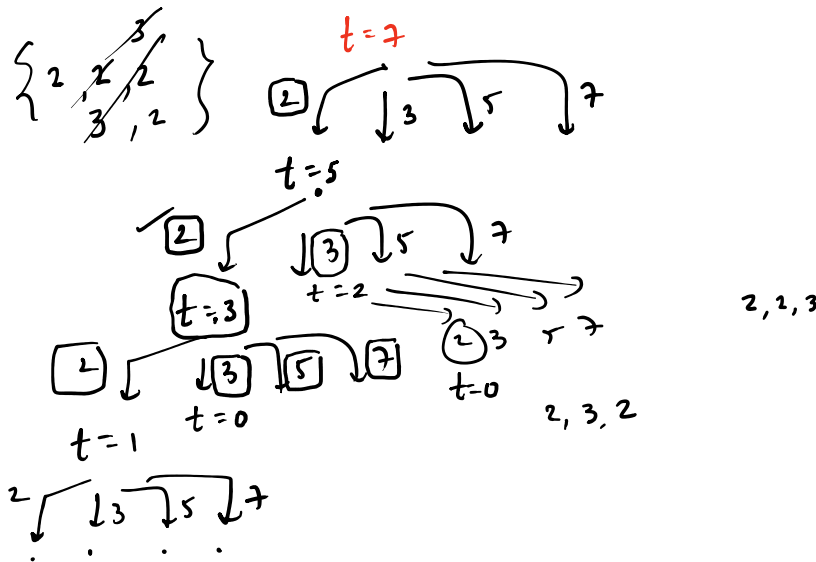
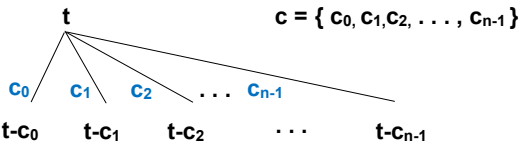
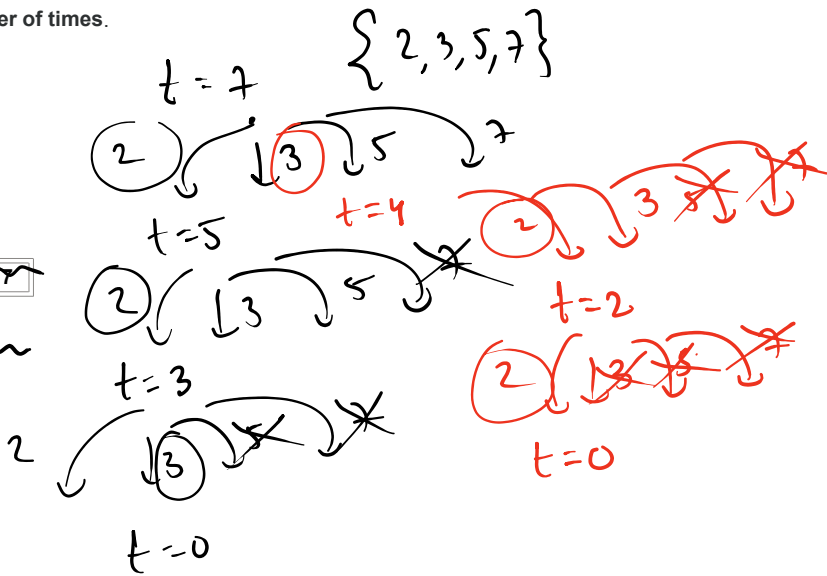
~~~~~

~~~~~

~~~~~

~~~~~

~~~~~



Combination Sum

Given an array of **distinct** integers **candidates** & a target integer **target**, design an algorithm to find all the **unique combinations of candidates** that sum up to the target. You may print the combinations in **any order**.

The **same** number may be chosen from candidates an **unlimited number of times**. Two combinations are said to be unique if the frequency of at least one of the chosen numbers is different.

Input

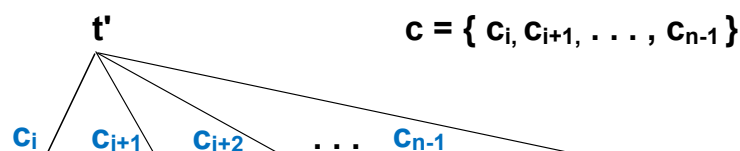
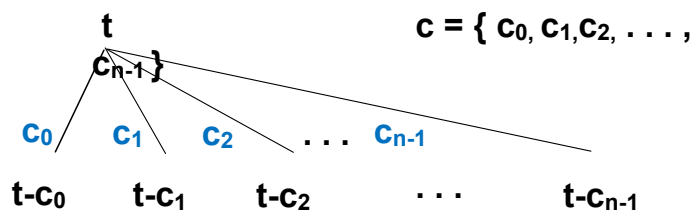
target = 7	0	1	2	3
candidates	2	3	5	7

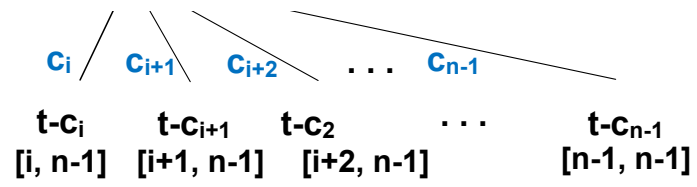
Output

2	2	3	2	5	7
---	---	---	---	---	---



2, 2, 3 ✓
 2, 3, 2 ✗
 3, 2, 2 ✗
 2, 5 ✓
 5, 2 ✗
 7 ✓





[HW] Combination Sum II

<https://leetcode.com/problems/combination-sum-ii/description/>

Ladder's Problem

Consider a ladder with **n** steps, a person is standing at the bottom of the ladder and wants to reach to its top. Design a recursive algorithm to **count** the number of ways the person can reach the **nth step** i.e. ladder's **top** from its **bottom** such that at each step the person can take a jump of size ranging b/w **one** to **k** steps.

Example

Input : **n = 4**, **k = 3**

Output : **7**

Explanation : [1, 1, 1, 1]

[1, 1, 2]

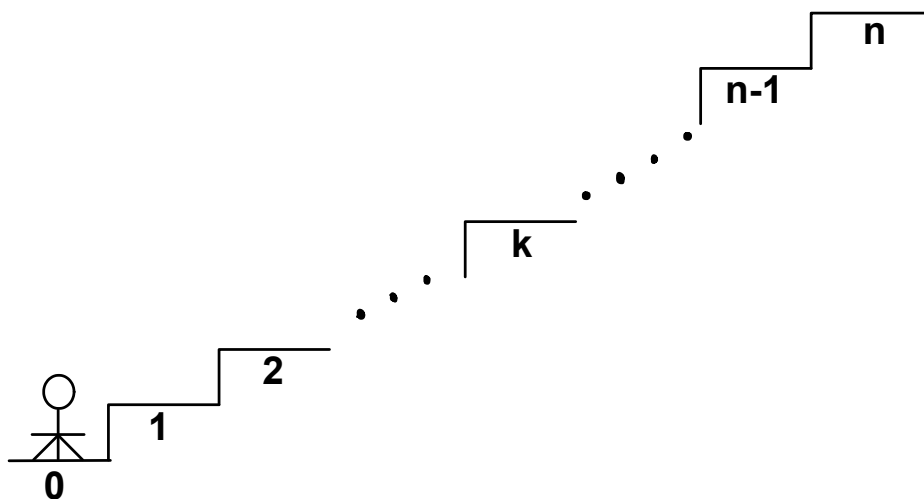
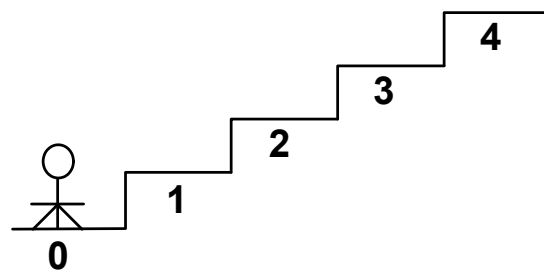
[1, 2, 1]

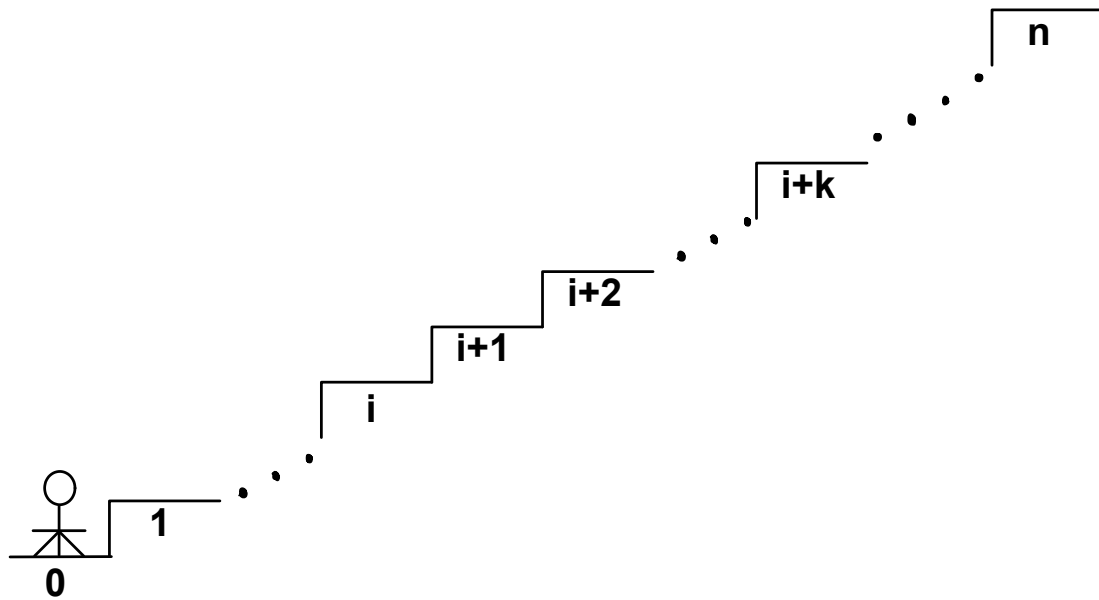
[2, 1, 1]

[2, 2]

[1, 3]

[3, 1]





Recursive Case

Let $f(i)$ be a fn. which computes the no. of ways a person can go from the i^{th} step to the n^{th} step of the ladder then,

$$f(i) = \sum_{j=1}^k f(i+j) ; i+j \leq n$$

Base Case

Rat in a Maze I

Rat in a Maze I

Given a $M \times N$ maze with some **blocked** cells (marked with **X**), and a **rat** that wants to go from the $(0, 0)^{\text{th}}$ cell (source) to the $(M-1, N-1)^{\text{th}}$ cell (destination) design a recursive algorithm to check if there **exists a path** to from the source to destination cell such at any point of time the rat can move one step either in the right direction or the down direction.

Example

Input :

0	0	0	0
0	→	0	X
0	↓	0	X
0	X	→	0

Output : True

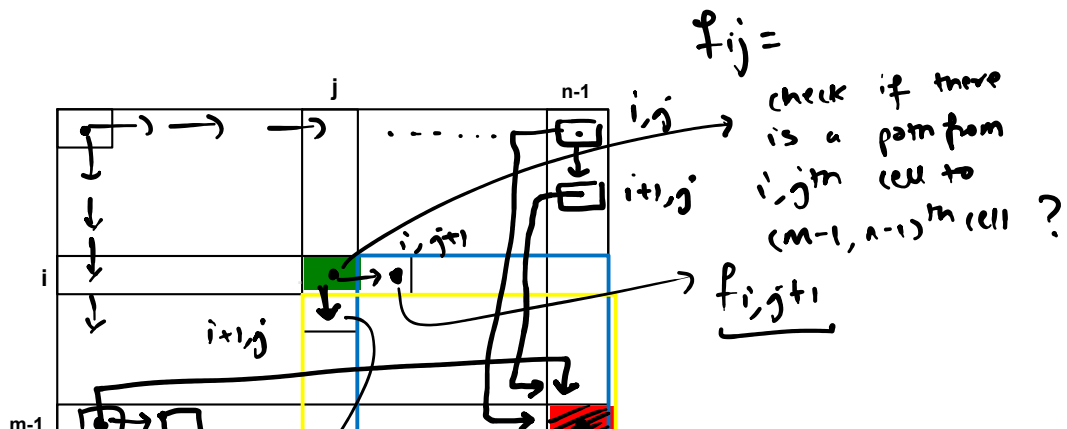
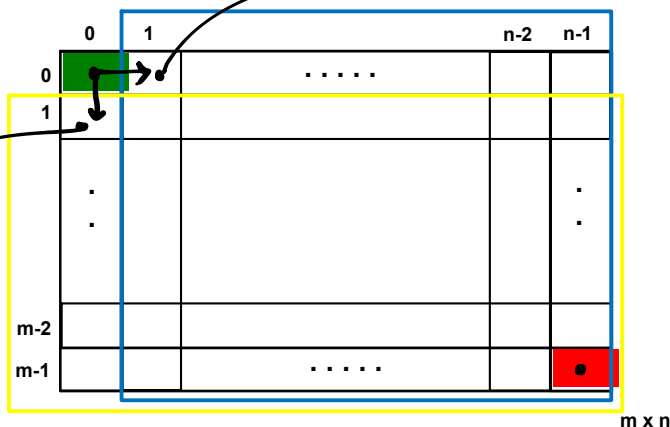
→ X or ↓

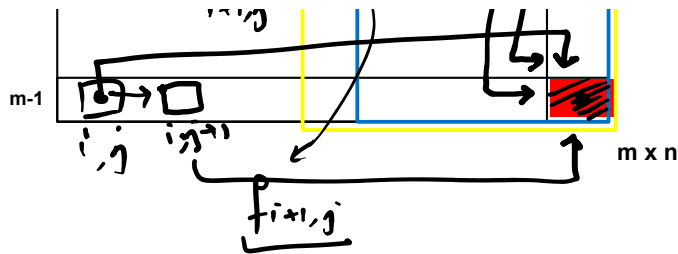
check if rat can go from $(0,0)$ to $(m-1,n-1)$?

ask your friend to check if rat can go from $(0,1)$ to $(m-1,n-1)$? X (bool) =

ask your friend to check if rat can go from $(1,0)$ to $(m-1,n-1)$?

y (bool)





init $i=0$
 $j=0$

rec-case

$$f_{ij} = f_{i,j+1} \text{ or } f_{i+1,j}$$

$$f_{ij} = f_{i+1,j} \quad \text{when } j == n-1 \text{ and } i \neq m-1$$

$$f_{ij} = f_{i,j+1} \quad \text{when } i == m-1 \text{ and } j \neq n-1$$

$$\text{when } i == m-1 \text{ and } j == n-1$$

base case

true

Rat in a Maze II

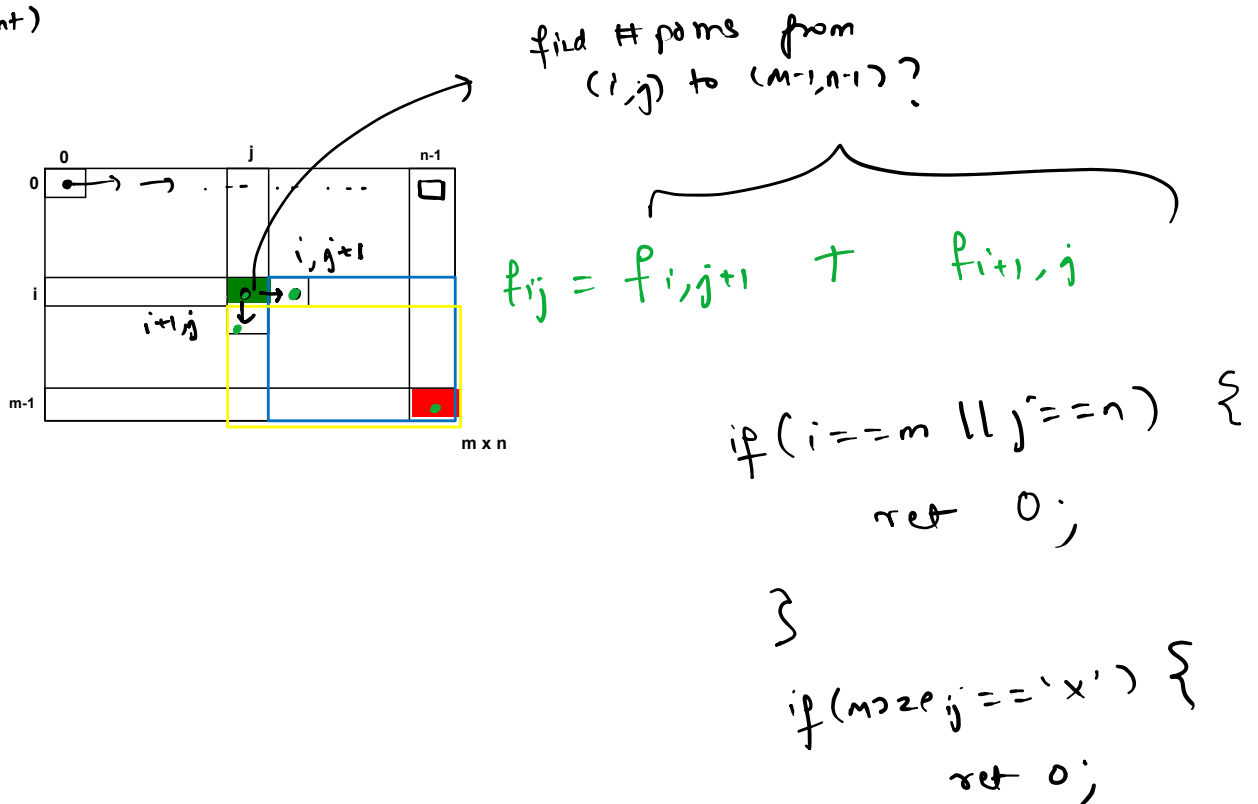
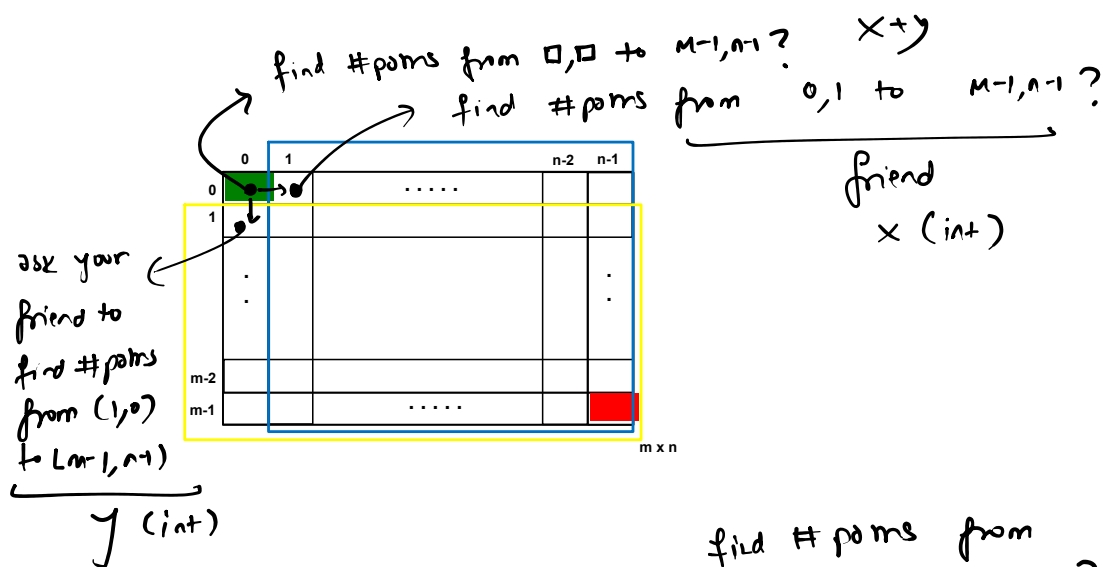
Given a $M \times N$ maze with some **blocked** cells (marked with **X**), and a **rat** that wants to go from the $(0, 0)^{\text{th}}$ cell (source) to the $(M-1, N-1)^{\text{th}}$ cell (destination) design a recursive algorithm to **count** the number of paths that exists to reach destination cell from the source cell such at any point of time the rat can move one step either in the right direction or the down direction.

Example

Input :

0	0	0	0	0
0	0	0	X	0
0	0	0	0	X
0	X	0	0	0

Output : 3



```

    }
    {
        if ( i == m-1 and j == n-1 ) {
            ret 1;
        }
    }

```


Rat in a Maze III

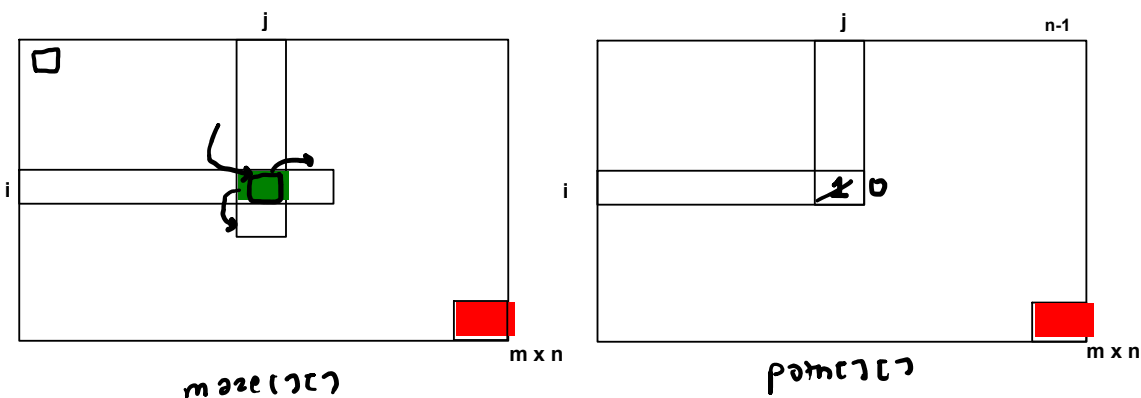
Rat in a Maze III

Given a $M \times N$ maze with some **blocked** cells (marked with **X**), and a **rat** that wants to go from the $(0, 0)^{\text{th}}$ cell (source) to the $(M-1, N-1)^{\text{th}}$ cell (destination), design a recursive algorithm to **generate all the paths** that exists to go from source cell to the destination cell such that at a point of time the rat can move one step either in the **right** direction or the **down** direction.

Example

Input :

0	0	0	0	1	0	0	0	1	0	0	0	1	1	0	0
0	0	X	0	1	0	0	0	1	1	0	0	0	1	0	0
0	0	0	X	1	1	1	0	0	1	1	0	0	1	1	0
0	X	0	0	0	0	1	1	0	0	1	1	0	0	1	1



$path[i][j] = '2'$
 $f(i, j+1)$
 $f(i+1, j)$
 $path[i][j] = '0'$

$if (i == m-1 \text{ and } j == n-1) \{$
 $\quad \text{ret};$
 $\}$

$if (maze[i][j] == 'X') \{$
 $\quad \text{ret};$
 $\}$

$if (i == m-1 \text{ and } j == n-1) \{$
 $\quad \text{print path}$
 $\quad \text{ret};$
 $\}$

$\}$





Rat Chases Its Cheese

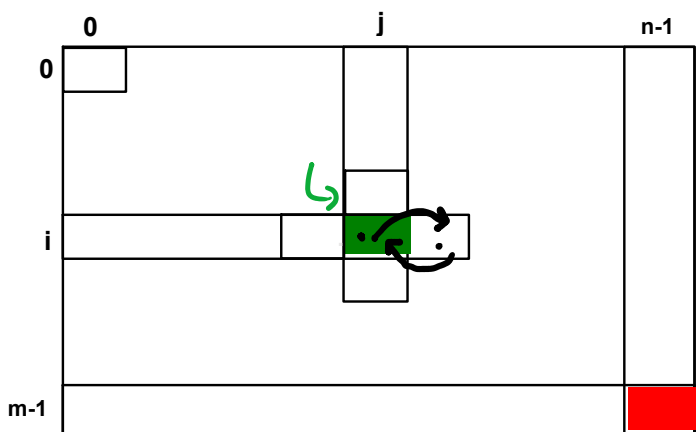
Rat Chases Its Cheese

Given a $M \times N$ maze with some **blocked** cells (marked with **X**), and a **rat** that wants to go from the $(0, 0)^{\text{th}}$ cell (source) to the $(M-1, N-1)^{\text{th}}$ cell (destination), design an algorithm to **find the unique path** that the rat has to take to go from source cell to the destination cell such that at a point of time the rat can move one step in the **left, right, up, or the down** direction.

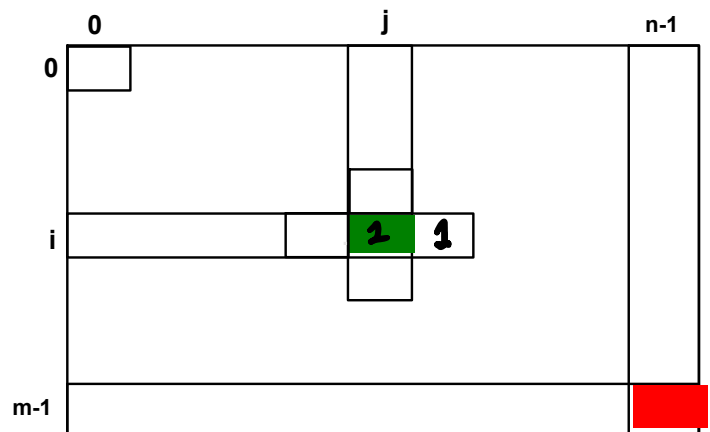
Example

Input :

0	X	0	0	1	0	0	0
0	0	0	X	1	1	0	0
X	0	X	0	0	1	0	0
X	0	0	X	0	1	1	0
X	X	0	0	0	0	1	1



$m \times n$



$m \times n$

[HW] Unique Paths III

<https://leetcode.com/problems/unique-paths-iii/>

Word Search

Given an $m \times n$ grid of characters board and a string **word**, return *true* if **word exists in the grid**. The word can be constructed from letters of **sequentially adjacent** cells, where adjacent cells are *horizontally* or *vertically* neighbouring. The same letter cell may not be used more than once.

Example

word = ABCCED

A	B	C	E
S	F	C	S
A	D	E	E

word = SEE

A	B	C	E
S	F	C	S
A	D	E	E